“Tilmann’s Cijfer” generator – design

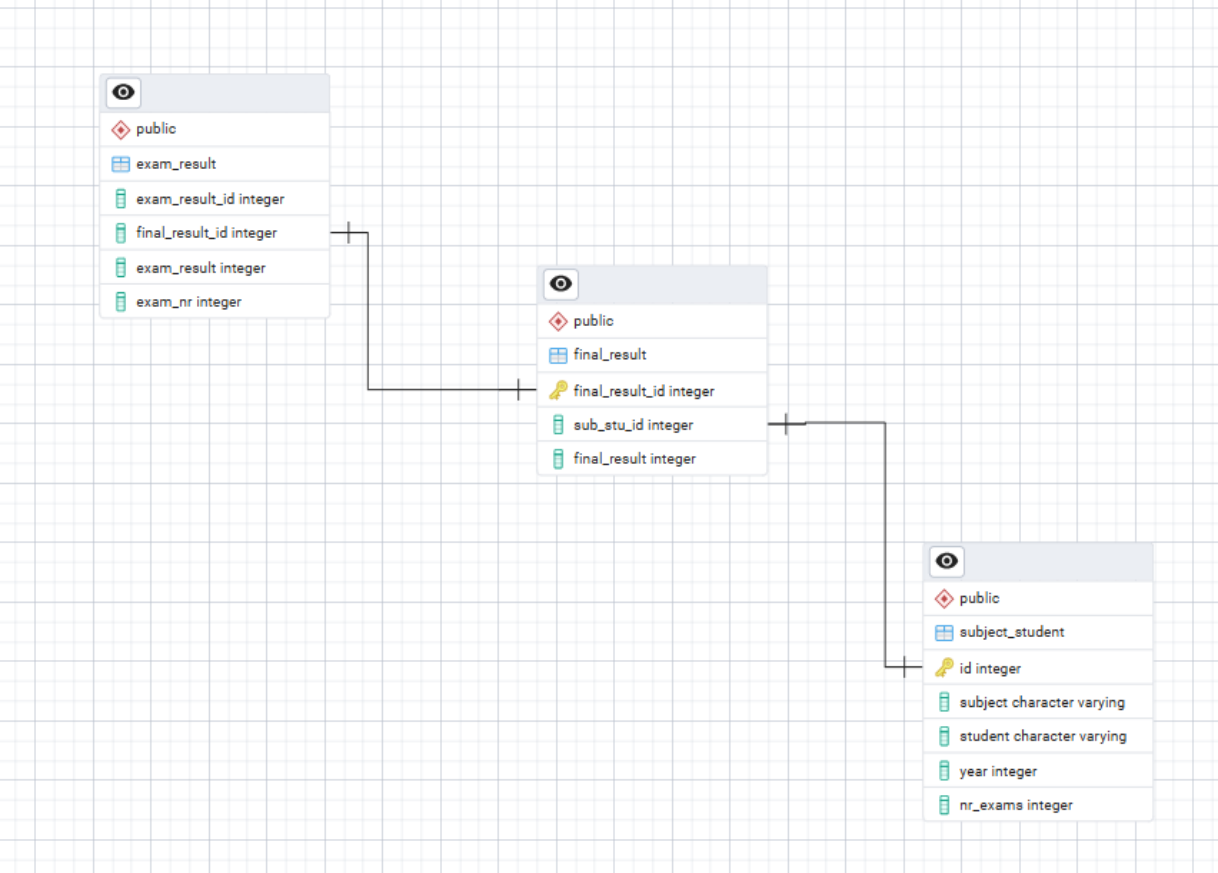
Introduction

Giving marks to students costs a lot of time and pain. For this an automatic solution is needed. Therefore, the overall IT architect (Tilmann K.) has designed a solution. The solution must be built within the ROS2 architecture. The application design is described in this document.
Following this design a prototype application is has to be developed.

Application database

All essential information has to be stored in a database. The database that has to be used is MariaDB (mysql) or Postgres.

The following three tables exist for essential data. These tables need to be implemented.



Subject_student table:

Contains the students that were following a subject in a certain year. For these students the final subject results and exam results must be created by the application.
The field exam_nr contains how many exams the students had to do when following subject.

Note: For the prototype there will be no user interface for the input. This table will be prefilled and updated directly through database tools.

Final result table:

The final result table contains the final mark of the student that he/she followed in a certain year.

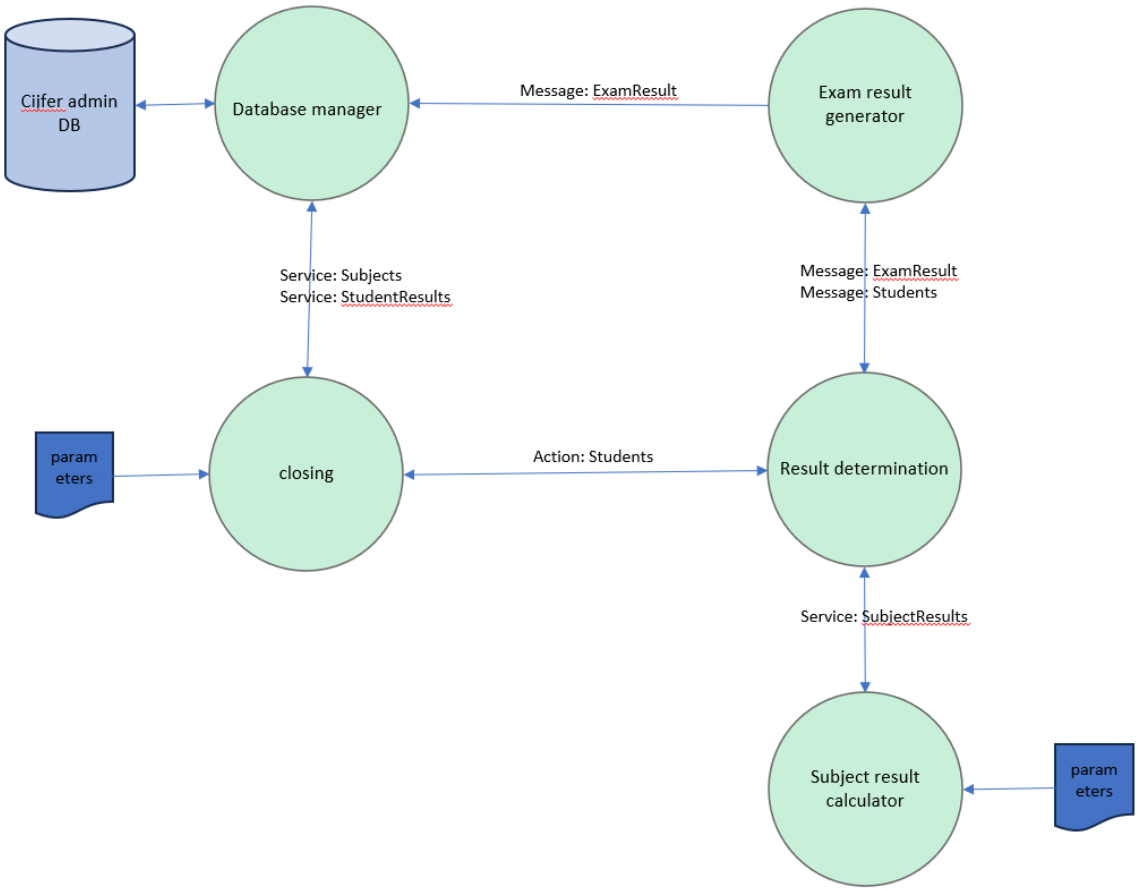
Note: This table will be filled by the application

Exam_result :
The table contains all exams that a student has taken for a subject.

Note: This table will be filled by the application

Nodes

ROS2 design



The solution design has following ROS2 **nodes**

- database manager
- closing
- Result determination
- subject result calculator
- exam result generator

The **interfaces** of the design are:
Msg:

- ExamResult
- Students

Srv:

- Subjects

- SubjectResults
- StudentResults

Action:

- students

Exam result generator

The exam result generator node subscribes to a message (msg: Students) that telling for which student/subject combinations the node must randomly produce exam results. The random generation will stop for a student/subject combination when it has generated exactly the number of exams results that is expected for this student/subject combination. All exam results are published (msg: ExamResult).
(parameters: timing of random generation process)

Result determination

The result determination node receives an action (action:Students) that contains all students/subject combinations belonging to a certain year for which result must be determined. This list is published (msg: students). The node subscribes to the message that sends the random generated exam results (msg:ExamResults). For each student/subject/year combination from the action message it collects the generated results. In case all results are captured for one record (collected exams = nr_exams) the node sends a request to the subject result calculator to get the result for the subject. The node continuously sends feedback and will return the action result

subject result calculator

The exam result calculator receives service requests (srv:SubjectExams) to calculate the final grade for an exam. This result is returned to the requester. The logic is as follows:

1 exam = 100%

2 exam = 30% 70% (default - parameter a,b)

3 exam = 20% 30% 50% (default – parameter a,b,c)

4 exam= 10%20%30%40% ((default – parameter a,b,c,d)

5 exam or more (counting equally)

Closing

The closing node sends for each year that needs to be closed an action request (action: Students). It should be possible to cancel a year closing action. In case the action stopped and also all data that is created in the database needs to be cleaned up (clean up service)

The closing node sends a request (src:subjects) to the databasemanager node to retrieve all subjects of a year (src: Subjects).

(parameters: closing years).

Database manager

- The database manager node subscribes to the ExamResult message (msg: ExamResult) and saves these in the database.
- From the closing node the database manager node receives service requests (src:Students). It returns students/subject combinations (all combinations belonging to a certain year)
- From the closing node the database manager node receives service requests (src:StudentsResults) to insert all students results belonging to a year. It returns students/subject combinations (all combinations belonging to a certain year)

Interfaces

All interfaces have to a standard header

msg: ExamResults

- student
- subject
- exam result
- exam_sequence_nr

msg: Students

array of

- student
- subject
- year
- nr_exams

msg: SubjectExams

- student
- subject
- year
- array of exam results

src : Subjects

request

- year
- subject

result

array of

- student
- subject
- year
- nr_exams

src : Clean up

request

- year
- timestamp

result

- SQL message

Action: Students

Goal

year

array of

- student
- subject
- nr_exams

feedback:

array of

- student
- subject
- year
- received_exams
- nr_exams

result:

array of

- student
- subject
- year
- result